

- **Generatorul (svbt_data_in_generator):** Creează și randomizează obiecte de tip `svbt_packet`. Prin polimorfism (folosind o metodă virtuală `copy()`), generatorul aplică constrângeri specifice în funcție de testul curent (ex. `delay == 0` pentru Stress

Test sau `bad_parity == 1` pentru Parity Test) și le trimite către BFM printr-un mailbox.

- **BFM / Driver (`svbt_data_in_bfm`):** Acționează ca un Master pe magistrala de intrare. Respectă cu strictețe protocolul router-ului: conduce semnalele pe frontul descrescător (`negedge`), așteaptă numărul de cicli specificat de variabila `delay` și monitorizează semnalele de `busy` și `reset` înainte de a aserta `packet_valid`. De asemenea, poate simula abandonarea pachetelor la jumătatea transmisiei prin flag-ul `mid_packet_drop`.
- **Monitoarele (`svbt_monitor_in`, `svbt_monitor_out`):** Monitorizează pasiv magistralele pe frontul crescător (`posedge`) pentru acuratețe la nivel de ciclu de ceas.
 - *Monitorul de intrare* reconstruiește pachetul din octeții trimiși succesiv (header, payload, parity) și îl trimite către Scoreboard.
 - *Monitorul de ieșire* supraveghează semnalul `ready`. Când acesta devine 1, extrage valoarea finală de pe magistrala `rdata` (valabilă un singur ciclu) și o trimite la Scoreboard.
- **Scoreboard-ul (`svbt_scoreboard`):** Este "creierul" verificării. Preia pachetele de la ambele monitoare și acționează ca un *GOLDEN MODEL* (Model de Referință) pentru a verifica dacă DUT-ul a procesat datele corect.

2. Strategia de Verificare (Checkere & Scoreboard)

Arhitectura și Rolul Scoreboard-ului Scoreboard-ul din acest mediu de verificare nu este un simplu comparator de date, ci un **Model de Referință Predictiv (Golden Model)**. Rolul său este de a menține starea virtuală a memoriilor sclavilor și de a emula la nivel de software comportamentul logic așteptat din partea hardware-ului.

Mecanismul de funcționare:

- **Sincronizarea Interfețelor:** Utilizează două canale de comunicație (Mailbox-uri) pentru a prelua obiectele reconstruite de `monitor_in` (ceea ce primește router-ul) și `monitor_out` (răspunsul sclavului validat de semnalul `ready`).

Modelarea Memoriei: Scoreboard-ul stochează intern într-o matrice valorile pe 8 biți ale celor 3 sclavi (`slv_reg`) și o serie de flag-uri ce reprezintă starea de blocare a acestora (`slv_lock`).

Evaluarea Proactivă (Predictorul): Înainte de a evalua ieșirea, Scoreboard-ul inspectează datele de la intrare. Dacă o comandă adresează un sclav blocat (`lock = 1`) sau o adresă protejată, Scoreboard-ul recunoaște imediat pachetul ca fiind invalid și prezice un "Silent Drop". Această inteligență îi permite să treacă la următorul pachet din coadă fără a aștepta un răspuns inexistent de pe magistrala de ieșire, prevenind astfel blocarea mediului de verificare.

- **Validarea Tranzacțiilor:**
 - La o operație de **WRITE**, calculează intern suma logică XOR pe toți bytes din payload și actualizează registrul virtual al sclavului.
 - La o operație de **INC**, adună valoarea ultimului octet la valoarea curentă a sclavului.

- La o operație de **READ**, realizează o comparație strictă (Data Match Checker) între valoarea teoretică calculată și răspunsul (`rdata`) eșantionat de monitorul de ieșire. Orice diferență ridică un mesaj de eroare critic, incrementând contorul de erori al testului.
-

3. Plan de Testare și Analiza Acoperirii (Coverage)

Pentru a asigura validarea robustă a router-ului (SCR), planul de testare a fost structurat pentru a acoperi atât funcționalitatea de bază, cât și scenariile limită (corner cases) sau de stres. Au fost create clase de pachete derivate din clasa de bază `svbt_packet`, folosind mecanisme de polimorfism (`virtual function copy`) pentru a injecta constrângeri specifice fiecărui scenariu de test.

Teste Implementate

Nume Test	Obiectiv	Rezultat
test_basic	Verificarea fluxului fundamental de operații (scriere, citire, incrementare) pe Slave 0, cu temporizări moderate (15-20 tick-uri).	Pass
test_functional	Testarea exhaustivă a tuturor adreselor de slaves și a tuturor comenzilor (inclusiv reset/lock/unlock pe adresa 3).	Pass
test_stress	Verificarea stabilității DUT-ului forțând injecția de pachete back-to-back (înlăturarea pauzelor dintre tranzacții prin setarea <code>delay == 0</code>).	Pass
test_random	Explorarea unui spațiu de stări extins, cu payload-uri mari, generare complet aleatoare și testarea robusteții la stimuli imprevizibili.	Fail (Vezi Bug RTL 3)
test_err_addr	Verificarea imunității DUT-ului (comportamentul de <i>Silent Drop</i>). Trimiterea intenționată de comenzi normale de date către adresa rezervată 3.	Pass
test_mid_drop	Simularea unei căderi a conexiunii pe magistrala de intrare. Semnalul <code>packet_valid</code> este coborât în mijlocul transmiterii payload-ului.	Pass (Vezi Bug RTL 2)

Nume Test	Obiectiv	Rezultat
test_parity_err	Generarea de pachete cu un octet de paritate corupt intenționat pentru a declanșa refuzul procesării din partea DUT-ului.	Pass (Fals, Vezi Bug RTL 1)

Analiza Acoperirii (Coverage)

- **Ce a fost acoperit:** Prin implementarea unui `covergroup` (`cg_router_traffic`) în Monitorul de Intrare, s-a obținut o acoperire completă a adreselor sclavilor (`SLAVE_0`, `SLAVE_1`, `SLAVE_2`, `SECURITY`) și a tipurilor de comenzi (`WRITE`, `READ`, `INC`, `SPE`). S-a atins *cross-coverage*-ul între comenzi și adrese, excluzând coșurile ilegale gestionate prin `ignore_bins`. Au fost stimulate mărimi variate de pachete (`SMALL`, `MEDIUM`, `LARGE` până la 15 bytes).

Necunoscută+ 2

- **Ce NU a fost acoperit (Limitări):** Nu a fost implementată validarea temporală automată folosind SystemVerilog Assertions (SVA). Verificarea numărului exact de cicli de ceas consumați de router între primirea pachetului și generarea răspunsului `ready` a fost realizată doar prin inspecție vizuală pe formele de undă (Waveforms). De asemenea, testele nu injectează zgomot asincron (glitches) pe semnalul de reset.

4. Analiza Bug-urilor

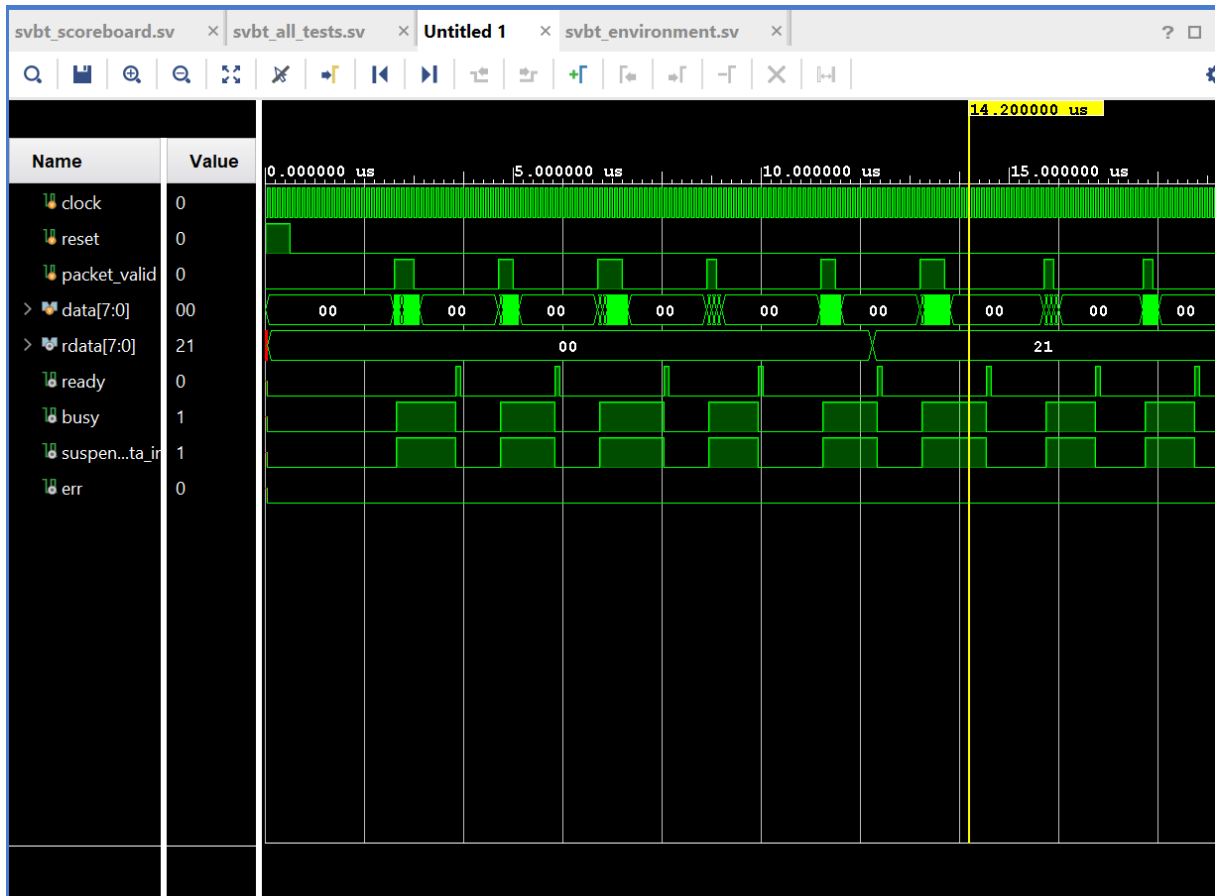
Pe parcursul dezvoltării testbench-ului și a rulării testelor, mediul de verificare a scos la iveală mai multe probleme. Acestea au fost împărțite în defecte de proiectare a hardware-ului (RTL) și vulnerabilități inițiale ale mediului de testare (remediate).

A. Bug-uri identificate în Design (Hardware / DUT)

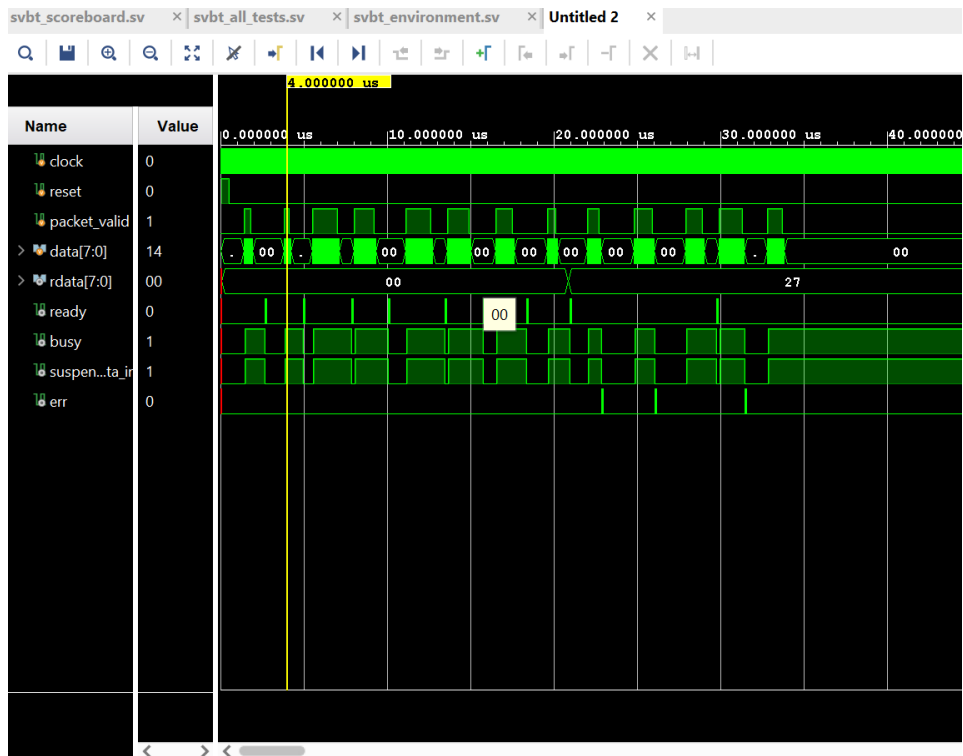
1. **Ignorarea erorilor de paritate:** Testbench-ul a generat intenționat pachete cu câmpul de paritate corupt. Deși protocolul impune refuzul pachetelor alterate, logurile din Scoreboard confirmă că DUT-ul a acceptat tranzacțiile și a scris datele eronate în registrele sclavilor. DUT-ul prezintă o vulnerabilitate critică privind integritatea datelor.
2. **Ignorarea protocolului `packet_valid` (Mid-Drop):** În scenariul de cădere a conexiunii (`test_mid_drop`), deși semnalul `packet_valid` a fost coborât de master la mijlocul payload-ului, router-ul a ignorat această întrerupere. Designul așteaptă rigid scurgerea cicli-lor de ceas dictați de câmpul `length`, fără a monitoriza activ linia de validare, finalizând o operație cu date inexistente.
3. **Blocajul magistralei (Bus Hang) la adrese neimplementate:** Rularea `test_random` a relevat o lipsă a verificării de limite (Bounds Checking) în router. Când DUT-ul a decodat accesul la Sclavul 3 (Sclav neimplementat ca block de memorie pe bus-ul de

B. Ajustări realizate în Testbench (Program Bug-fixes)

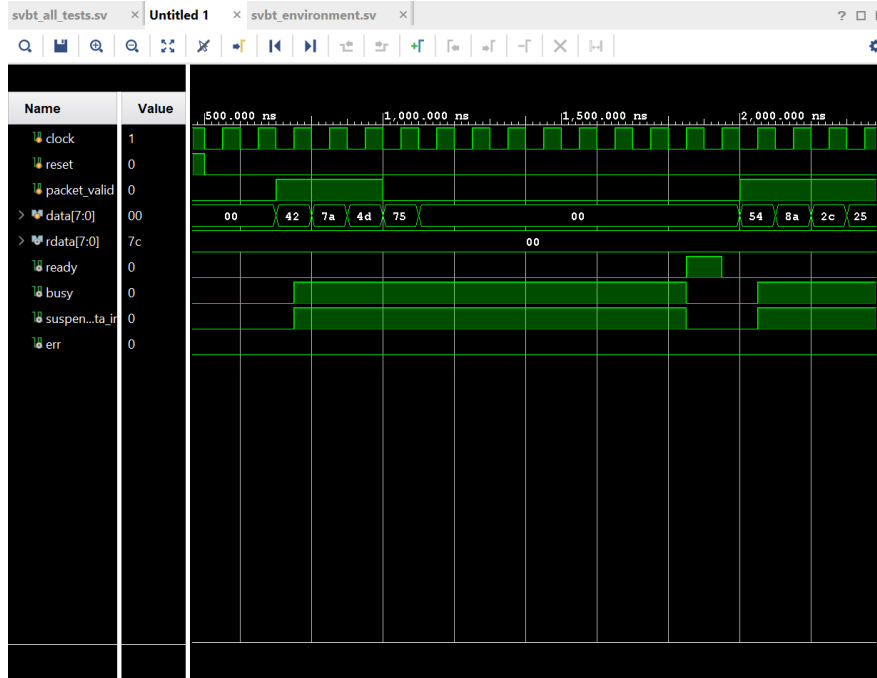
- ### FORME DE UNDA:



Basic test



Random test



Stress test